Patent Management System Implementation Report

Branch: 11-PMS
Specification set: Assignment 7, PMS v1.2/v1.3
Status: Backend lifecycle implementation present; final verification is blocked by the local PostgreSQL service and Ruff issues. The client pages remain placeholders.

Architecture and design

PMS is a Django domain module under modules.pms, registered in DOMAIN_MODULES and mounted at /api/v1/pms/. It follows the module layering contracts for API, services, selectors, models, and domain logic. Cross-module references use integer IAM user IDs; contracts.py exposes plural-ID lookups to other modules. The domain state machine owns status transitions, services enforce lifecycle rules and write audit/notification records, selectors scope application reads, and DRF serializers/views form the API boundary.

Features present in the code

- Application submission, unique application numbers, review assignment and decisions, revision/resubmission, withdrawal, and appeals.
- Inventor ownership shares, consent/agreement records, and withdrawal consent.
- PDF size/malware validation metadata and document version tracking.
- Budget decisions and escalations, attorney panel/assignment/response, patentability assessments, and legal advice.
- Prior-art search/attachment, external filing/grant tracking, office actions and responses, and maintenance decisions/actions.
- Licensing requests, audit trail, report aggregates, notification outbox, and deadline monitoring services.
- PMS-specific role/permission declarations and navigation manifest entries.

The integration currently records external filing and notification state in PMS; it does not perform legal decisions, actual payments, or patent-office submission. Search is a PMS-supported adapter/stub rather than a live patent database integration.

Models and relationships

The migration creates PatentApplication and related records: CoInventor, ApplicationDocument, DirectorReview, PatentBudget, AttorneyAssignment, AttorneyPanel, PatentabilityAssessment, PriorArtReference, OfficeAction, PostGrantMaintenance, LicensingRequest, PatentAppeal, PatentLegalAdvice, PmsAuditTrail, PmsNotificationOutbox, PmsConfiguration, and PmsRoleAssignment. Applications own lifecycle records; budgets are one-to-one; actor references are scalar IAM IDs. 0001_initial.py is the only PMS migration currently present.

Workflows and business rules

Services and state transitions cover the documented submission/review/revision/appeal paths, attorney assignment, budget decisions, filing, office-action response (with and without applicant review), maintenance, and pre-filing withdrawal. Rule validation includes required fields, document constraints, reviewer justification, deadlines, assignment conflicts, ownership shares, and inventor consent. The test suite names and maps cases to CFR/BR/UC/BW identifiers, though coverage is selective rather than exhaustive.

Authorization and integrations

The registry declares PMS roles and permissions for applicant, Director/Dean Research, PCC admin, and attorney workflows. API endpoints use module grants and permission gates; application list/detail reads use the visible_applications selector. During continuation, revise_and_resubmit_application was corrected to check primary-applicant ownership at the service boundary, and an authorization regression test was added (CFR-PMS-008). Missing OpenAPI annotations were added for prior-art search, consent, reporting, renewal, and office-action actions. Office-action extension input now uses a bounded serializer instead of an unchecked integer conversion. The generated OpenAPI schema and permission manifest were refreshed.

Known authorization limitation: several nested resource endpoints retrieve records by ID in their services/views and do not consistently apply the application selector first. Director selector scope is also broader than assigned-only because domain membership is not currently modeled. These require security review before production use.

Files created and modified

- Added modules/pms/ (domain, models, migration, services, selectors, API, and tests).
- Added client/src/modules/pms/ routes and a placeholder page.
- Modified .importlinter, config/settings/base.py, and config/urls.py to register the module and API.
- Modified registry/permissions.json and generated docs/02-iam/permission-catalog.generated.md with PMS permissions/navigation.
- Added this report.
- Added one regression test in modules/pms/tests/test_authorization.py for cross-applicant revision denial.

No unrelated module implementation files were changed. Ruff formatting and safe autofixes were applied to the new PMS package after the project check reported lint issues.

Requirement coverage

Implemented in backend code: core application lifecycle; document and inventor records; review, budget, assignment, filing, office action, maintenance, licensing, appeal, prior-art, audit, and notification flows. The existing tests reference all major workflow families (UC-PMS-001 through UC-PMS-033), a selected subset of the 30 CFRs and 84 BRs, and the 16 basis/5 composite workflows.

Partially implemented:

- Report UC-PMS-024/030/031/032: current API returns basic status/domain aggregates; full dashboards, workload analytics, and workflow oversight are not present.
- UC-PMS-025: configuration and role-assignment models exist, but complete configuration/role administration endpoints are absent.
- UC-PMS-026: attorney panel listing/creation exists; full edit/deactivation and expertise lifecycle are incomplete.
- UC-PMS-016: deadline monitoring service exists but no scheduled-job registration/operation is supplied in this module.
- UC-PMS-002: status/history is represented by application state and audit rows, but there is no dedicated history endpoint.
- UI: all PMS routes currently render Placeholder; no applicant, reviewer, attorney, or admin workflow screens are implemented.
- External prior-art search and notification delivery depend on integration/deployment adapters; this repository does not provide a live external search connector.

Not implemented or not verified: complete requirement-by-requirement acceptance evidence; fully functional PMS client pages; scheduled task wiring; production-grade malware scanning and binary document storage; full configuration, exceptional-case, and analytics workflows. Because database-dependent tests could not initialize, these requirements are not considered successfully tested.

Tests and checks

Existing PMS tests were inspected before adding a single regression case. Test modules cover domain transitions, business rules, models, workflows, authorization, deadlines/notifications/audit, and API behavior.

Commands executed:

- make check failed at .venv/bin/ruff check .. Django manage.py check passed; makemigrations --check --dry-run reported no changes (with a warning that the PostgreSQL connection was unavailable). Ruff initially reported 227 PMS lint findings; after safe formatting/autofixes, 69 remained, primarily E501 long lines plus unused test variables and two regex warnings. Later checks in the Make target did not execute.
- ./.venv/bin/pytest -q modules/pms/tests 26 passed, 73 errors. Errors occurred while creating/connecting to the test database because PostgreSQL at 127.0.0.1:5432 was unavailable. There were no assertion failures reported; database-backed tests, including the new regression test, remain unverified.
- client checks were not run: client/node_modules is absent.
- lint-imports, no_cross_module_fk.py, contracts_are_plural.py, and nav_matches_routes.py passed.
- manage.py permission_manifest --check passed after restoring the PMS PERMISSIONS export expected by the manifest generator.
- make schema / manage.py spectacular --fail-on-warn passed after adding the missing PMS endpoint schemas.

Setup and rerun instructions

1. Start the project PostgreSQL service and configure DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, and DB_PORT for the local environment.
2. From Fusion-Integrated, run make migrate to apply migrations.
3. Run the backend checks with make check and the PMS tests with ./.venv/bin/pytest -q modules/pms/tests.
4. Install the client dependencies according to the repository lockfile, then run cd client && npm run typecheck && npm test && npm run build.

Readiness

The module is not ready to push as a completed assignment: database-backed tests have not run successfully, project lint still fails, and the client and several administration/reporting requirements remain partial. The schema and permission checks were rerun after the first make check; the full test suite was not rerun after the final schema/validation edits, so those edits also remain unverified by execution. The report records observed code and execution results rather than treating declared features as verified behavior.